SISTEMAS OPERATIVOS • UEES

Diseño de Solución Concurrente: Gestión de Laboratorio

Justificación del acceso concurrente, gestión de recursos compartidos y exclusión mutua en Python.

Autores
Eddy Lima & José Troya

Materia
UCOM418_81 Sistemas Operativos

Contextualización y Unidades de Ejecución

El sistema administra un laboratorio académico donde múltiples estudiantes solicitan estaciones de trabajo de forma simultánea.

Unidades de Ejecución (Hilos)

Cada estudiante se modela mediante la clase UsuarioThread. Múltiples hilos compiten al mismo tiempo por obtener asignación.

Múltiples Solicitudes
Peticiones simultáneas a la terminal libre.

Recursos Limitados
Número finito de PCs disponibles en el inventario.

El Recurso Compartido

¿Qué representa?

Estado físico y lógico de los equipos y registro centralizado de tiempos de uso.

¿Quiénes lo utilizan?

Los hilos de usuarios UsuarioThread para consultar, asignar y liberar terminales.

¿Por qué hay conflicto?

Acceso simultáneo genera condiciones de carrera y asignación doble de una PC.

Representación del Recurso en Atributos

OPCIÓN A

Tipo Básico (Entero)

Contador simple de computadoras disponibles.

```
pcs_disponibles = 10
```

Límitación: Carece de metadatos como ID de equipo o usuario.

OPCIÓN B

Tipo Complejo (Diccionario)

Estructura anidada con trazabilidad completa.

```
estado_pcs = {
   "pc_01": {"estado": "disponible", "usuario": None}
 }
```

Ventaja: Control detallado del inventario y asignaciones.

Diseño de Clases y Responsabilidades

LaboratorioManager

Responsabilidad: Gestor centralizado del recurso.

Atributo: estado_computadoras (Diccionario).

Atributo Sinc: lock (threading.Lock).

Métodos: asignar_computadora(), liberar_computadora().

UsuarioThread (threading.Thread)

Responsabilidad: Simular al alumno solicitando equipo.

Atributo: id_usuario.

Atributo Ref: gestor_lab (Instancia compartida).

Método: run() solicita, usa y libera.

Implementación en Código (Python)

```
# Representación Básica
class LabBasico: def __init__(self, total): self.disponibles = total def asignar(self): if self.disponibles > 0: self.disponibles -= 1 return True return False
```

class LabBasico: def __init__(self, total): self.disponibles = total def asignar(self): if self.disponibles > 0: self.disponibles -= 1 return True return False

```
# Representación Compleja
class LaboratorioManager: def __init__(self, pcs): self.pcs = {f"pc_{i}": {"estado": "disponible", "usr": None} for i in pcs} def asignar_computadora(self, usuario_id): for pc_id, info in self.pcs.items(): if info["estado"] == "disponible": info["estado"] = "ocupado" info["usr"] = usuario_id return pc_id return None
```

class LaboratorioManager: def __init__(self, pcs): self.pcs = {f"pc_{i}": {"estado": "disponible", "usr": None} for i in pcs} def asignar_computadora(self, usuario_id): for pc_id, info in self.pcs.items(): if info["estado"] == "disponible": info["estado"] = "ocupado" info["usr"] = usuario_id return pc_id return None

Análisis de la Sección Crítica

Instrucciones Críticas (Consulta - Validación - Modificación)

Buscar equipo libre (if info["estado"] == "disponible") y cambiar su estado (info["estado"] = "ocupado").

Inconsistencia Sin Sincronización

Asignación Doble: Dos hilos leen "disponible" simultáneamente en la misma PC antes de actualizarla. Ambos reclaman el mismo equipo.

Corrupción de Estado

Sobrescritura del ID de usuario asignado, dejando el registro en un estado inconsistente.

Mecanismo de Protección: Mutex

```
# Protección con threading.Lock
import threading class LaboratorioManager: def __init__(self, pcs): self.pcs = {...}
self.lock = threading.Lock() def asignar_computadora(self, usuario_id): with
self.lock: # Adquisición / Liberación for pc_id, info in self.pcs.items(): if
info["estado"] == "disponible": info["estado"] = "ocupado" info["usr"] = usuario_id
return pc_id return None
```

import threading class LaboratorioManager: def __init__(self, pcs): self.pcs = {...} self.lock = threading.Lock() def asignar_computadora(self, usuario_id): with self.lock: # Adquisición / Liberación for pc_id, info in self.pcs.items(): if info["estado"] == "disponible": info["estado"] = "ocupado" info["usr"] = usuario_id return pc_id return None

Justificación de Elección

Se elige **Mutex (Lock)** sobre semáforo para garantizar exclusión mutua estricta en la actualización de la estructura compleja.

Puntos de Control

El cerrojo se adquiere antes del bucle de validación y se libera al retornar la PC o salir del bloque with.

Conclusión y Selección de la Representación



Representación Compleja

Se concluye que el **Diccionario Estructurado** es la representación más adecuada para la gestión del laboratorio.

Trazabilidad Detallada

Permite asociar qué usuario específico utiliza cada máquina y desde cuándo.

Control Granular

Facilita operaciones de mantenimiento e identificación de terminales individuales.

Integridad con Mutex

Protegido con threading.Lock, ofrece alta velocidad en memoria compartida sin perder detalle.

¿Preguntas o Comentarios?

Gracias por su atención. Presentación basada en el diseño de concurrencia para el Laboratorio de Computación.